Skooly — System Design Summary

Project Summary for Thesis Supervisor

---

1. Project Overview

Skooly is a multi-tenant school management SaaS platform developed as part of my thesis work. Unlike traditional single-school systems, Skooly is designed as a **networked platform** where schools, teachers, and families can connect across the ecosystem. The platform aims to provide a single, unified application for managing academic operations, people, scheduling, and billing—with a roadmap toward cross-school features such as teacher mobility and student transfer and inter-school activities.

1.1 Core Capabilities

- **Five user roles**: Admin, Teacher, Student, Parent, and System Admin—all in one unified app with role-based views and dashboards
- **Academic operations**: Academic year management, classes, grades, subjects, curriculum, scheduling, lessons, exams, assignments, attendance, and results
- **People management**
- **Subscription billing**: Stripe integration for plan-based pricing per school

1.2 What Differentiates Skooly

Aspect	Skooly
Model	Multi-tenant SaaS, subscription-based (Stripe)
Teacher mobility	Teacher marketplace (planned): teachers can work at multiple schools, discoverable by schools
Student mobility	Student transfer between schools (planned); parent access across children's schools
Curriculum	Optional country-based templates (France, USA, UK, Morocco, IB/Cambridge)
Schedule changes	Teacher-initiated requests (time change, swap); admin approves with conflict checks
Cross-school	Roadmap: sports leagues, alumni network, group purchasing, shared transport
Tech stack	Next.js 14, Prisma, PostgreSQL, Server Actions, App Router

---

2. Technology Stack

Layer	Technology	Purpose
Frontend/Backend	Next.js 14, App Router	Single-page app with server-side rendering, RSC, Server Actions

Layer	Technology	Purpose
Database	PostgreSQL via Prisma ORM	Relational data, migrations, type-safe queries
Authentication	Custom JWT	HttpOnly cookie,
Payments	Stripe	Subscription plans, checkout, webhooks
Media	Cloudinary	Image uploads (profiles, etc.)

3. Architecture & Design Decisions

3.1 High-Level Architecture

The application follows a layered architecture:

- **Client layer**: Browser, React components
- **Next.js layer**: Pages (RSC), Server Actions, API routes, middleware
- **External services**: Stripe, Cloudinary
- **Data layer**: Prisma ORM → PostgreSQL

3.2 Key Architectural Decisions

1. **Hybrid data access**: Server Actions handle most CRUD operations (subjects, classes, lessons, exams, etc.); REST API routes are used for auth, Stripe checkout, webhooks, and some read-only endpoints (announcements, subscription plans).

2. **JWT-based authentication**: Custom auth with `auth_token` HttpOnly cookie. Token: JWT (HS256, 7-day expiry). Middleware verifies the token on each request and applies role-based redirects.

3. **Multi-tenant model**: Row-level isolation via `schoolId` on all school-scoped tables. No separate database per tenant. Middleware enforces `user.schoolId === path schoolId`; `/system/*` restricted to system_admin.

4. **Enrollment as source of truth**: `StudentEnrollmentHistory` is the canonical record of a student's class and grade for an academic year. `Student.classId` and `Student.gradeId` are derived and kept in sync on enroll/unenroll. Academic activities (attendance, assignments, schedules) use enrollment data to determine which students belong to a class.

3.3 Authentication & Onboarding Flows

Flow	Description
New school admin	User creates account + school in one step at `/create-school`
Existing user	Sign-in at `/sign-in` → redirect by role (admin, teacher, student, parent, system_admin)
Invited user	Admin sends invite → user receives email with link → `/invite/[token]` → sets password → Auth + profile created

Flow	Description
Bulk onboarding	CSV upload for teachers, students, parents; optional "Set your password" emails

3.4 Role-Based Access Control

- `/schools/[schoolId]/*`: User must have `schoolId === schoolId` (no cross-school access)
- `/system/*`: Only `system_admin`
- Unauthenticated: Redirect to `/sign-in`

---

4. Core Domains & Features

4.1 People Management

Feature	Implementation
List pages	Teachers, students, parents with search, filters, pagination
CRUD	Create, update, delete via Server Actions
Invite system	Send invite, accept at `/invite/[token]`, resend, bulk invite, invite on enrollment
Bulk import	CSV upload for students, teachers, results; validation and error reporting

4.2 Academic Structure

Entity	Purpose
AcademicYear	Bounds a school year; classes, curriculum, enrollments are scoped to it
Grade	School-level grade (e.g. Grade 1, Grade 5, Kindergarten)
Class	Section within a grade for an academic year (e.g. "5A", "Grade 3-B")
Subject	School-level subject (e.g. Math, French, Science)
Curriculum	Links academic year + grade + subject; defines what is taught in each grade
StudentEnrollmentHistory	Records which class/grade a student is in for a given academic year

Curriculum enforcement: Lessons must reference subjects that belong to the class's grade curriculum. Optional country-based templates (France, USA, UK, Morocco, IB/Cambridge) for schools to load pre-built curricula.

4.3 Academic Activities

Activity	Description
Lessons	Core scheduling unit: subject, class, teacher, room, day, time); conflict checks (teacher overlap, class overlap, room overlap, teacher availability)

Activity	Description
Exams	Formal assessments linked to lessons; role-based filtering
Assignments	Homework/tasks linked to lessons; student visibility based on enrollment
Attendance	Per lesson and date; status: Present, Absent, Late
Results	Scores for exams/assignments; bulk import supported

4.4 Scheduling

View	Features
Admin	FullCalendar (timeGridWeek, timeGridDay) with drag-and-drop and resize; create/edit lessons; filters (class, teacher); teacher availability overlay; conflict checks
Teacher	View schedule; request TIME_CHANGE or SWAP; cancel pending requests
Student	Class schedule for active academic year (from enrollment)
Parent	Schedules for each enrolled child

Schedule change requests: Teachers request time changes or teacher swaps; admins approve or reject with conflict checks. Room double-booking is prevented across all flows.

---

5. API Structure

Area	Routes	Purpose
Auth	`/api/auth/sign-in`, sign-out, set-token, me	Authenticate, set cookie, current user
Academic	`/api/schools/[schoolId]/academic-years`, curriculum, enrollments	CRUD for academic structure
Resources	`/api/schools/[schoolId]/rooms`, announcements	Rooms, announcements
Billing	`/api/schools/[schoolId]/subscriptions/*`, `/api/subscription-plans`	Stripe checkout, plans
System Admin	`/api/system_admin/subscription-plans`, school-subscriptions	Manage plans, school subscriptions
Webhooks	`/api/webhooks/payment`	Stripe webhooks

---

6. Current Progress — Implemented Features

The following parts of the application are **already developed**:

6.1 Core Platform

- Multi-tenant school management

- Five user roles with role-based access control
- JWT-based authentication (custom, HttpOnly cookie)
- Route protection and school isolation via middleware
- Subscription management (Stripe integration)

6.2 People Management

6.3 Academic Structure

- Academic year management
- Classes, Grades, Subjects
- Curriculum management (per academic year, per grade)
- Student enrollment history

6.4 Scheduling & Academic Activities

- Lessons and scheduling (FullCalendar, drag/drop, resize)
- Teacher availability management
- Schedule change requests (teacher-initiated time change/swap, admin approve/reject)
- Room/venue management
- Exams, Assignments, Results
- Attendance tracking

6.5 Other

- Events and Announcements
- Basic analytics (charts for attendance, counts, finance)

---

7. Planned Features (Roadmap)

7.1 Core Missing Features

- Messaging & communication (WebSockets/SSE, direct and group messaging)
- Fee management and billing (student fees, payment tracking)
- Library management
- Transportation management
- Advanced reporting and analytics
- Gradebook and transcript generation
- Notification system
- Parent-teacher conference scheduling

7.2 Cross-School Ecosystem (Planned)

- **Teacher marketplace**: Multi-school support, discovery, contact flow, structured availability
- **Student transfer**: Simple transfer model; parent multi-school access; initiation by new school, parent, or old school
- Cross-school sports leagues & tournaments
- Alumni network across schools

- Group purchasing & bulk buying
- Shared transportation routes
- Cross-school parent networks
- Shared extracurricular activities

---

9. External Integrations

Service	Usage
Stripe	Subscription plans, checkout, webhooks (`checkout.session.completed`, `customer.subscription.*`)
Cloudinary	Image uploads (profile, etc.)
PostgreSQL	Primary database via Prisma